Laboratorio di Calcolo, Esercitazione 1, 15-16 ottobre 2026

Canale Q-Z, Docenti: Shahram Rahatlou, Alessandra Betti

Lo scopo di questa esercitazione è familiarizzare con la postazione Linux e scrivere i primi programmi in C. Nella **Parte 1** imparate a muovervi nel terminale, a organizzare file e cartelle e a usare l'editor `emacs`; compilate ed eseguite un primo programma. Nella **Parte 2** scrivete programmi che leggono numeri con `scanf()`, li elaborano e li stampano con `printf()`. I listati da digitare sono dati nel testo.

▶ Postazione di lavoro

Fate login con username `lcsrXXX`, dove `XXX` è il numero del vostro gruppo (ad esempio `lcsr042`); l'account è condiviso dal gruppo e i file restano fino alla fine del corso; la password vi viene comunicata in aula. Cercate nel menu delle applicazioni la voce *Terminale* e apritela. A fine lavoro fate il *logout*.

▶ Nozioni utili

`pwd`	dove sono	`mkdir A B`	crea le cartelle `A` e `B`
`ls`	elenca i file (`-l`, `-a`, `-R`)	`touch f`	crea il file vuoto `f`
`cd d`	entra nella cartella `d`	`cp f d/`	copia `f` dentro `d`
`cd ..`	sale di un livello	`mv f d/`	sposta `f` dentro `d`
`cd`	torna alla home	`mv f g`	rinomina `f` in `g`
`cat f`	mostra il file `f`	`rm f`	cancella `f` (**per sempre!**)
`man ls`	manuale (`q` per uscire)	`emacs f &`	apre `f` lasciando libero il terminale

Con il tasto **Tab** il terminale completa i nomi, con la freccia ↑ richiama i comandi già scritti, con **Ctrl-C** interrompe un programma.

Emacs. Usate `emacs` nella sua versione grafica: col mouse si fa tutto. In alto c'è il menu (`File`, `Edit`, `Options`, `Buffers`, `Tools`, `Help`) e sotto la barra dei pulsanti. Da sinistra: nuovo file, apri file, apri cartella, chiudi il file (la `X`: chiude il file, non esce da emacs), `Save` (grigio finché non ci sono modifiche da salvare), `Undo`, forbici (taglia), due fogli (copia), appunti (incolla), lente (cerca). Taglia e copia si attivano dopo aver selezionato del testo col mouse. Per uscire da emacs: menu `File`, ultima voce, oppure la `X` della finestra. **Attenzione:** in emacs `Ctrl-C` e `Ctrl-V` *non* copiano e incollano come nelle altre applicazioni: usate i pulsanti o il menu `Edit`. Ciò che non è salvato si perde: salvate spesso. Le scorciatoie da tastiera sono comode e vale la pena impararle, ma con calma: `C-x` significa tenere premuto Ctrl e premere `x`.

`C-x C-s`	salva il file	`C-x u`	annulla l'ultima modifica
`C-x C-c`	esce da emacs	`C-g`	interrompe un comando iniziato per errore
`C-x C-f`	apre un altro file	`C-w`, `C-y`	taglia, incolla il testo selezionato
`C-s`	cerca un testo		(di nuovo `C-s`: il successivo)

PARTE 1: la postazione e la shell

▶ 1.1 Orientarsi nel terminale

1. Con `pwd` scoprite dove vi trovate: qual è il percorso completo della vostra home? Con `ls`, `ls -a` e `ls -l` guardate cosa contiene: che differenze ci sono?
2. Con `man ls` leggete a cosa serve l'opzione `-R` (uscite con `q`).
3. Creare una cartella `ESER1` con `mkdir ESER1`: conterrà tutto il materiale di questa esercitazione. Entrateci con `cd ESER1` e osservate come cambia il prompt.

▶ 1.2 File di testo con emacs

1. Nella cartella `ESER1` aprite con `emacs spesa.txt &` un nuovo file e scrivete tre righe (la vostra lista della spesa). Salvate con il pulsante `Save` e chiudete emacs dal menu `File` (ultima voce).
2. Fate lo stesso per altri due file: `ricetta.txt` (la vostra ricetta preferita) e `idee.txt` (tre idee per il fine settimana).
3. Controllate con `ls` e `cat spesa.txt` che i tre file ci siano e contengano ciò che avete scritto.

▶ 1.3 Un albero di cartelle

1. Nella cartella `ESER1` create due cartelle `A` e `B` **allo stesso livello** (cioè `B` non dentro `A`).
2. Dentro `A` create le sottocartelle `A1`, `A2`, `A3` e dentro `B` le sottocartelle `B1`, `B2`, `B3`. Suggerimento: `mkdir A/A1 A/A2 A/A3` crea tre cartelle con un solo comando.
3. Con `ls -R` verificate di aver ottenuto questa struttura:

```
ESER1
|-- A
|   |-- A1
|   |-- A2
|   `-- A3
|-- B
|   |-- B1
|   |-- B2
|   `-- B3
|-- idee.txt
|-- ricetta.txt
`-- spesa.txt
```

▶ 1.4 Spostare, copiare, rinominare

1. Con `mv` spostate `ricetta.txt` dentro la cartella `A1`.
2. Con `cp` copiate `spesa.txt` dentro `A2`: l'originale deve restare in `ESER1`.
3. Con `mv` spostate `ricetta.txt` da `A1` a `B2` *senza entrare* in `A1` (usate il percorso `A/A1/ricetta.txt`).

4. Spostate `idee.txt` in `B3`. Poi entrate in `B3` con `cd` e, sempre con `mv`, cambiate il nome del file in `idee-v2.txt`.

5. Tornate in `ESER1` e controllate con `ls -R` dove si trova ciascun file. Con `cat` verificate che i contenuti non siano cambiati.

Attenzione: se la destinazione di `mv` o `cp` è il nome di un file già esistente, quel file viene sovrascritto senza avvisare.

▶ 1.5 Il primo programma in C

1. Entrate nella cartella `B1` e con `emacs scacchiera.c &` scrivete il programma seguente (rispettate le maiuscole, i punti e virgola e le virgolette). Salvate con il pulsante `Save`.

```c
#include <stdio.h>

int main() {
  printf("# . # . # . # .\n");
  printf(". # . # . # . #\n");
  printf("# . # . # . # .\n");
  printf(". # . # . # . #\n");
}
```

Listato 1: Programma `scacchiera.c`

2. Compilate con `gcc -Wall -o scacchiera.exe scacchiera.c` ed eseguite con `./scacchiera.exe`. L'opzione `-Wall` chiede al compilatore tutti gli avvisi; il `./` dice alla shell che il programma è nella cartella corrente (senza, risponde `command not found`). Se il compilatore protesta, leggete il messaggio (file, riga e colonna dell'errore), correggete in `emacs`, salvate e compilate di nuovo. Per vedere un errore vero, cancellate apposta un `;`, compilate e leggete il messaggio; poi rimettetelo.

3. Scrivete un programma `presentazione.c` che stampi tre righe: il vostro nome, il vostro corso di laurea e il titolo del vostro libro preferito. Compilatelo ed eseguitelo.

4. **Facoltativo.** Salvate ciò che il programma stampa in un file, invece che sullo schermo, con la redirezione `>` (la vedrete meglio più avanti):

```
./presentazione.exe > presentazione.txt
```

   Controllate con `cat presentazione.txt`. Rieseguendo il programma con `>>` al posto di `>`, il testo viene aggiunto in fondo al file invece di sostituirlo: provate. Poi spostate il file in `A3` con `mv` e verificate con `ls -R`, da `ESER1`, che ci sia.

PARTE 2: programmi che leggono e calcolano

In questa parte **ricopiate** i listati dal foglio al terminale: lo scopo è abituarsi al ciclo scrivere, salvare, compilare, eseguire. Non serve aver capito ogni riga: i dettagli li vedrete a lezione nei prossimi giorni. Rispettate punteggiatura, maiuscole e spazi, e leggete con attenzione i messaggi del compilatore se qualcosa non va.

▶ 2.1 Da Celsius a Fahrenheit

1. Tornate in `ESER1`, create la cartella `PROG` con `mkdir PROG` ed entrateci: ci metterete i programmi di questa parte.

2. Con `emacs convert.c &` scrivete il programma seguente e salvatelo.

```c
#include <stdio.h>

int main() {
  /* conversione da Celsius a Fahrenheit */
  double tc, tf, conv, offset;
  conv = 5. / 9.;
  offset = 32.;
  printf("inserisci temperatura in Celsius: ");
  scanf("%lf", &tc);

  tf = tc / conv + offset;

  printf("Temperatura Fahrenheit: %5.2f gradi\n", tf);
}
```

Listato 2: Programma `convert.c`

3. Compilate con `gcc -Wall -o convert.exe convert.c` ed eseguite con `./convert.exe`.

4. Verificate che il risultato sia corretto provando valori di cui conoscete la risposta: l'acqua bolle a 100 °C e il ghiaccio fonde a 0 °C. Quanto dovrebbe dare −40 °C?

5. Scrivete un nuovo programma `fahr2cels.c` che converta da Fahrenheit a Celsius, partendo da una copia di `convert.c` (`cp convert.c fahr2cels.c`) e modificandola. Cambiate sia la formula sia i messaggi stampati. Ricordate che $T_C = (T_F - 32) \cdot 5/9$. Verificate con almeno tre valori.

▶ 2.2 La libreria matematica

1. Con `emacs matem.c &` scrivete il programma seguente e salvatelo.

```c
#include <math.h>
#include <stdio.h>

int main() {
  /* uso della libreria matematica e dei descrittori di formato */
  double x, y;
  int n;

  printf("Inserisci un numero reale x: ");
  scanf("%lf", &x);

  printf("Inserisci un numero reale y: ");
  scanf("%lf", &y);

  printf("Inserisci un intero n: ");
  scanf("%d", &n);

  // stesso numero con descrittori diversi
  printf("%f + %f = %f\n", x, y, x + y);
  printf("%5.3f + %5.3f = %5.3f\n", x, y, x + y);
  printf("%5.3g + %5.3g = %5.3g\n", x, y, x + y);

  printf("potenza x^n\n");
  printf("%f ^ %d = %g\n", x, n, pow(x, n));

  printf("x come angolo in gradi: calcolo di cos(x)\n");
  y = (x / 180.) * M_PI;
  printf("cos(%f gradi) = %f\n", x, cos(y));
}
```

Listato 3: Programma `matem.c`

2. Questo programma usa funzioni della libreria matematica (`pow`, `cos`), quindi la compilazione richiede l'opzione `-lm` alla fine: `gcc -Wall -o matem.exe matem.c -lm`.

3. Eseguite il programma più volte con numeri diversi per `x`, `y` e `n`, ad esempio `4`, `12`, `-1.232322`, `56.2e3`, `-1.87e-12`. Osservate le tre righe della somma: come cambia il modo in cui è scritto lo stesso numero con `%f`, `%5.3f` e `%5.3g`? Non serve ancora spiegarlo: i descrittori di formato li vedrete a lezione nei prossimi giorni.

4. *Facoltativo:* provate a togliere `-lm` dal comando di compilazione e leggete il messaggio d'errore.